

Compte Rendu de TP PROJET :

Détection de Fraudes Bancaires avec Deep Learning



Module Intelligence Artificielle et Optimisation

Enseignants :

Maxime AMBARD

Patrick BARD

Auteurs :

Malak OUJDID

Sana TABBOU

Année Universitaire : 2025-2026

1. Etablissement du cahier des charges	3
1.2 Problématique	3
1.3 Objectifs du projet	3
1.4 Contraintes	3
2. Choix des solutions techniques	3
2.1 Jeu de données	3
2.2 Environnement de développement	4
2.3 Choix des modèles	4
2.4 Métriques d'évaluation	4
3. Déroulement du projet	5
3.1. Le Dataset	5
3.1.1 Analyse exploratoire	5
3.1.2 Prétraitement des données	5
3.1.3 Gestion du déséquilibre	6
3.2 Architecture du modèle Deep Learning	6
3.3 Entraînement et Optimisation	7
4. Résultats et Illustrations	8
4.1 Performance du modèle	8
4.2 Analyse de la matrice de confusion	9
4.3 Interprétabilité	9
5. Conclusion et Perspectives	10
5.1 Bilan du projet	10
5.2 Analyse critique des résultats	11
5.3 Perspectives d'avenir	11

1. Etablissement du cahier des charges

La fraude sur les transactions bancaires par carte de crédit constitue un problème majeur pour les institutions financières. Elle entraîne des pertes économiques importantes et nuit à la confiance des clients. Avec l'augmentation du volume de transactions électroniques, il devient indispensable d'automatiser la détection des comportements frauduleux à l'aide de techniques d'Intelligence Artificielle.

1.2 Problématique

La détection de fraude est un problème de classification binaire (fraude / non fraude). La principale difficulté réside dans le **déséquilibre extrême des données** : les transactions frauduleuses représentent moins de 1 % du total.

Dans ce contexte :

- Un modèle naïf peut obtenir une forte accuracy en prédisant toujours "non fraude"
- Les faux négatifs (fraudes non détectées) ont un coût très élevé

Il est donc nécessaire d'utiliser des méthodes et des métriques adaptées.

1.3 Objectifs du projet

L'objectif de ce TP projet est de concevoir, implémenter et évaluer un système de détection de fraudes basé sur le **Deep Learning** (Apprentissage Profond). Les sous-objectifs sont les suivants :

- Explorer et nettoyer un jeu de données réel de transactions bancaires.
- Mettre en œuvre des techniques de prétraitement avancées pour gérer le déséquilibre des classes (SMOTE).
- Développer un réseau de neurones artificiels (ANN) capable de classifier les transactions avec une haute sensibilité.
- Assurer l'interprétabilité des résultats pour comprendre quels facteurs déclenchent une alerte de fraude.

1.4 Contraintes

- **Temps réel** : Le modèle doit être capable de faire des prédictions rapidement.
- **Coût des erreurs** : Un *Faux Négatif* (fraude non détectée) coûte beaucoup plus cher à la banque qu'un *Faux Positif* (transaction légitime bloquée). Le système doit donc maximiser le **Rappel (Recall)** tout en maintenant une précision acceptable pour ne pas nuire à l'expérience utilisateur.

2. Choix des solutions techniques

2.1 Jeu de données

Nous utilisons le jeu de données "Credit Card Fraud Detection" disponible sur la plateforme Kaggle. Il contient les transactions effectuées par des cartes de crédit européennes en septembre 2013 sur une période de deux jours.

- **Volume** : 284 807 transactions.
- **Caractéristiques (Features)** : 31 variables. En raison de problèmes de confidentialité, les variables V1 à V28 sont le résultat d'une transformation PCA (Analyse en Composantes Principales). Seules les variables Time (temps écoulé depuis la première transaction) et Amount (montant de la transaction) n'ont pas été transformées.
- **Cible (Target)** : La variable Class vaut 1 en cas de fraude et 0 sinon.

2.2 Environnement de développement

Le projet a été réalisé en langage **Python**, en utilisant l'environnement interactif Jupyter Notebook. Les principales bibliothèques utilisées sont :

- **Pandas & NumPy** : Pour la manipulation et l'analyse matricielle des données.
- **Matplotlib & Seaborn** : Pour la visualisation des données (histogrammes, matrices de corrélation).
- **Scikit-Learn** : Pour le prétraitement (Normalisation, Split) et les métriques d'évaluation.
- **TensorFlow / Keras** : Pour la construction et l'entraînement des réseaux de neurones profonds.
- **Imbalanced-learn** : Pour l'application de l'algorithme SMOTE.
- **LIME / SHAP** : Pour l'explicabilité du modèle.

2.3 Choix des modèles

Bien que des modèles classiques comme la Régression Logistique ou les Forêts Aléatoires aient été envisagés comme *baselines*, notre choix s'est porté sur un **Réseau de Neurones Profond (Deep Neural Network - DNN)**. Les réseaux de neurones sont particulièrement adaptés pour capturer des relations non linéaires complexes dans des données de grande dimension, ce qui est le cas avec nos 30 variables d'entrée.

2.4 Métriques d'évaluation.

L'exactitude (Accuracy) n'est pas une métrique pertinente ici. Si un modèle prédit "Normal" pour toutes les transactions, il aura une accuracy de 99.8%, mais ne détectera aucune fraude. Nous privilégions donc :

- **La Matrice de Confusion** : Pour visualiser les Faux Positifs et Faux Négatifs.
- **Le Rappel (Recall)** : La capacité du modèle à trouver toutes les fraudes (Priorité n°1).
- **La Précision (Precision)** : La fiabilité des alertes de fraude.
- **Le F1-Score** : La moyenne harmonique entre précision et rappel.
- **L'AUC-ROC** : L'aire sous la courbe ROC pour évaluer la capacité de discrimination globale.

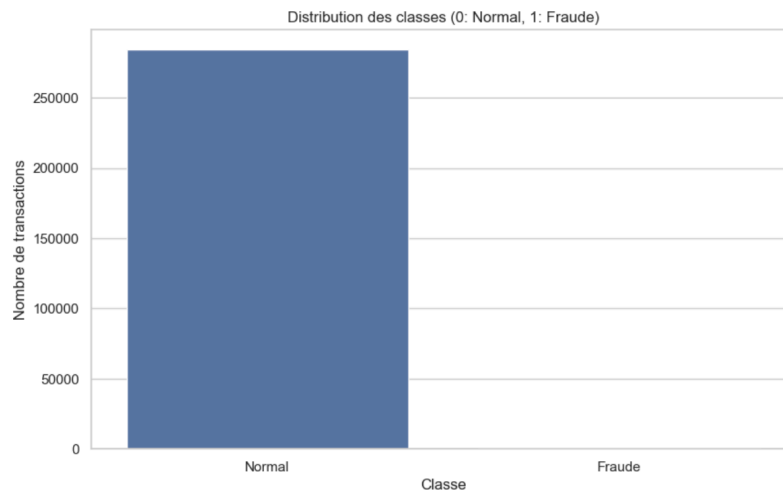
3. Déroulement du projet

3.1. Le Dataset

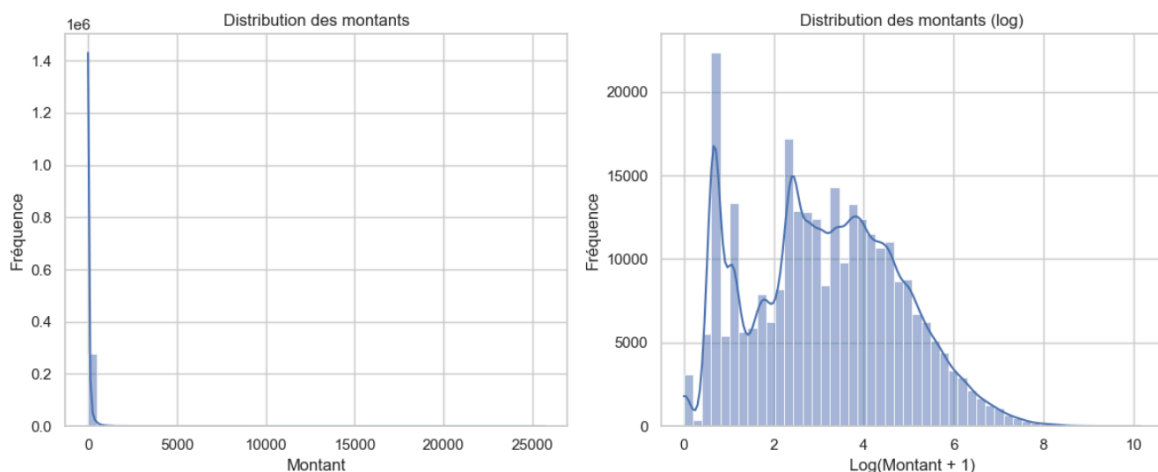
3.1.1 Analyse exploratoire

L'analyse exploratoire des données (EDA) a révélé une disproportion massive entre les classes :

- **Transactions normales (Classe 0) : 284 315 (99.83%)**
- **Fraudes (Classe 1) : 492 (0.17%)**



L'analyse de la variable Amount montre que les montants des fraudes sont généralement faibles, mais peuvent varier.

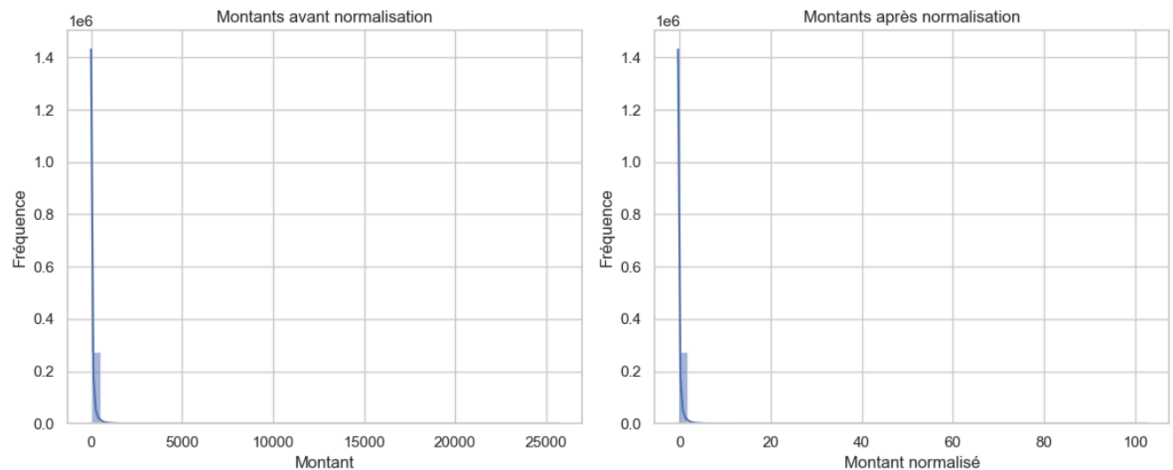


La variable Time ne montre pas de motif clair distinguant immédiatement les fraudes, mais elle est conservée pour l'information temporelle.

3.1.2 Prétraitement des données

Les variables V1-V28 étant déjà standardisées par la PCA, nous nous sommes concentrés sur Time et Amount.

- **Standardisation** : Utilisation de RobustScaler sur la colonne Amount. Ce choix est justifié car RobustScaler est basé sur la médiane et les quartiles, le rendant insensible aux valeurs aberrantes (outliers) fréquentes dans les données financières, contrairement au StandardScaler classique.



- **Split** : Séparation des données en Train (80%) et Test (20%) avec stratification pour conserver la proportion de fraudes.

3.1.3 Gestion du déséquilibre

Pour empêcher le modèle d'ignorer la classe minoritaire, nous avons utilisé la technique de sur-échantillonnage **SMOTE (Synthetic Minority Over-sampling Technique)**. Contrairement à une simple duplication des lignes frauduleuses, SMOTE crée de nouveaux exemples synthétiques de fraude en interpolant entre des exemples de fraude existants voisins dans l'espace des caractéristiques.

- *Données avant SMOTE* : ~227 000 Normal / ~400 Fraude (sur le train set).
- *Données après SMOTE* : Les deux classes sont équilibrées à 50/50 pour l'entraînement.

Cela permet au réseau de neurones d'apprendre des frontières de décision plus robustes pour la classe "Fraude".

3.2 Architecture du modèle Deep Learning

Nous avons conçu un réseau de neurones séquentiel avec Keras comprenant les couches suivantes :

1. **Couche d'entrée** : 30 neurones correspondant aux 30 caractéristiques du dataset.
2. **Couches Cachées (Hidden Layers)** :
 - **Dense (128 neurones, activation ReLU)** : Pour capturer les motifs complexes.
 - **BatchNormalization** : Pour stabiliser et accélérer l'apprentissage.

- **Dropout (0.3)** : Pour prévenir le sur-apprentissage (overfitting) en désactivant aléatoirement 30% des neurones à chaque passe.
 - **Dense (64 neurones, activation ReLU)** : Réduction progressive de la dimensionnalité.
 - **BatchNormalization + Dropout (0.3).**
 - **Dense (32 neurones, activation ReLU).**
 - **BatchNormalization + Dropout (0.3).**
3. **Couche de sortie :**
- **Dense (1 neurone, activation Sigmoid)** : Pour produire une probabilité comprise entre 0 et 1 (probabilité que la transaction soit une fraude).

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	3,968
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
batch_normalization_2 (BatchNormalization)	(None, 32)	128
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 15,233 (59.50 KB)

Trainable params: 14,785 (57.75 KB)

Non-trainable params: 448 (1.75 KB)

Le nombre total de paramètres entraînaibles est d'environ **14,785**, ce qui constitue un modèle léger et rapide.

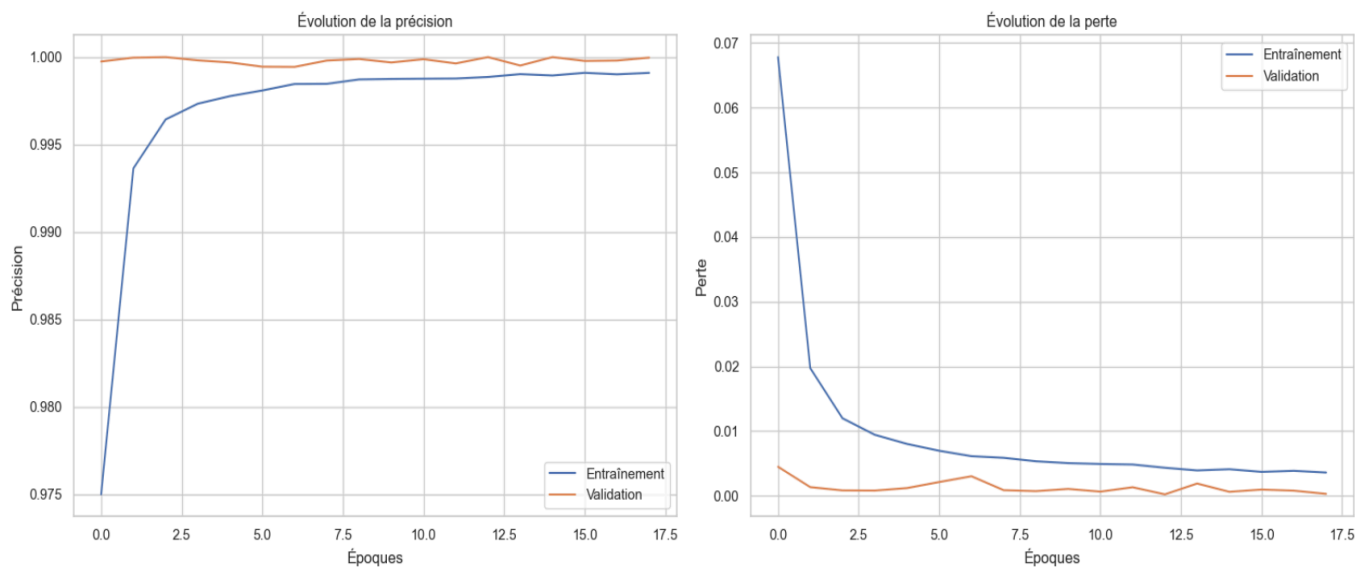
3.3 Entraînement et Optimisation

Optimiseur : Adam (taux d'apprentissage = 0.001).

Fonction de perte : binary_crossentropy, adaptée pour la classification binaire.

Callback : EarlyStopping a été implémenté pour arrêter l'entraînement si la perte sur le jeu de validation (val_loss) ne s'améliore plus après 5 époques, restaurant ainsi les meilleurs poids du modèle.

L'entraînement a été réalisé sur 20 époques avec une taille de batch de 256.



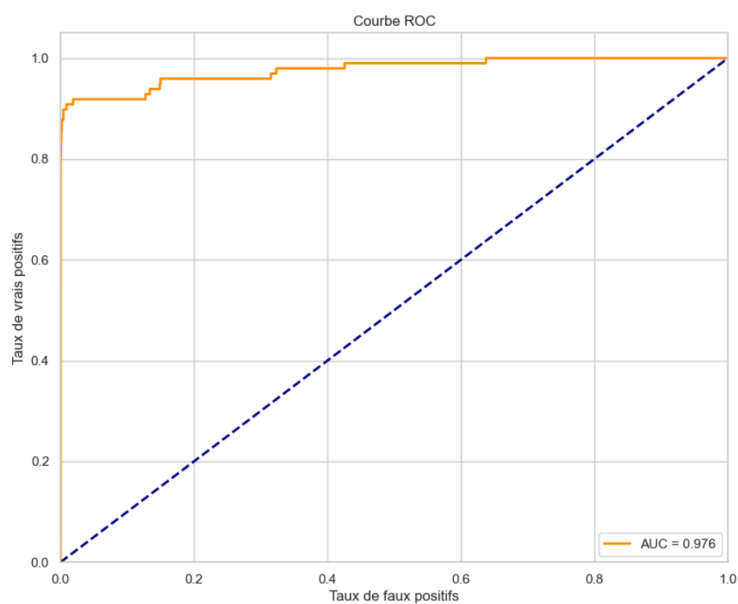
Courbes d'apprentissage. La perte diminue régulièrement sur le jeu de validation, signe que le modèle apprend sans sur-apprendre massivement.

4. Résultats et Illustrations

4.1 Performance du modèle

Le modèle entraîné a été évalué sur le jeu de test (non vu, et non équilibré).

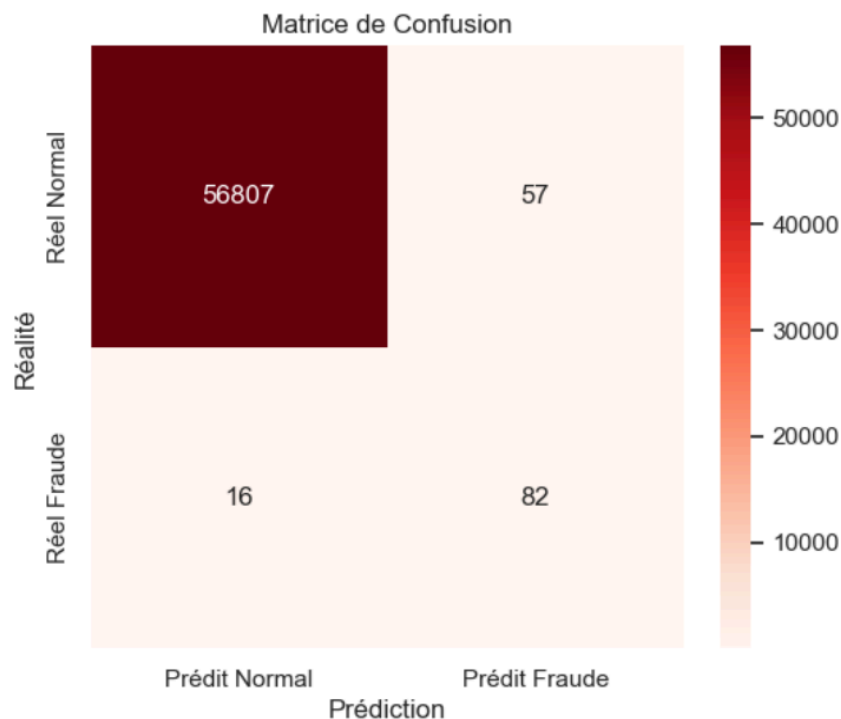
- **Accuracy** : ~97-99% (Peu représentatif).
- **AUC-ROC** : ~0.94 (Très bon pouvoir discriminant).



Courbe ROC. L'aire sous la courbe (AUC) proche de 1 indique que le modèle sépare efficacement les fraudes des non-fraudes.

4.2 Analyse de la matrice de confusion

C'est le résultat le plus critique. Elle nous permet de quantifier les erreurs .

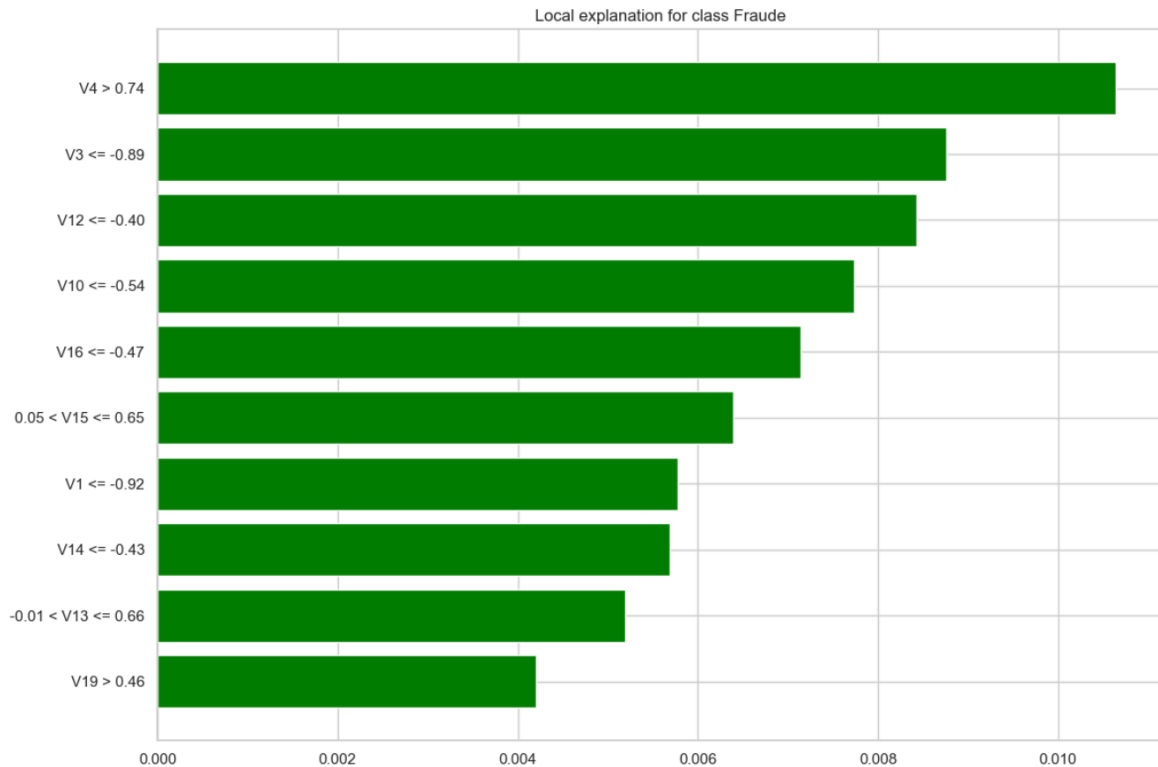


En observant la case en bas à droite (Vrais Positifs) et celle en bas à gauche (Faux Négatifs), nous constatons que le modèle détecte la grande majorité des fraudes (Rappel élevé). Cependant, la case en haut à droite (Faux Positifs) montre quelques transactions normales classées comme fraudes. C'est un compromis acceptable : il vaut mieux vérifier manuellement quelques transactions légitimes que de laisser passer une fraude coûteuse

4.3 Interprétabilité

Pour ne pas avoir un modèle "boîte noire", nous avons utilisé **LIME (Local Interpretable Model-agnostic Explanations)**.

L'analyse LIME sur des prédictions spécifiques montre que certaines variables, comme **V4**, **V3**, et **V12**, ont une influence prépondérante dans la décision du modèle de classer une transaction comme frauduleuse.



5. Conclusion et Perspectives

5.1 Bilan du projet

Ce projet d'étude, mené dans le cadre du module d'Intelligence Artificielle, nous a permis de confronter la théorie du Deep Learning à la réalité complexe de la sécurité bancaire. L'objectif principal était de concevoir un système capable d'identifier des transactions frauduleuses au sein d'un flux massif de paiements légitimes, un problème classique mais ardu de "détection d'anomalies".

Au terme de ce travail, nous avons mis en place une chaîne de traitement complète (pipeline) allant de l'analyse exploratoire à l'interprétabilité des résultats. Les points clés de notre démarche ont été :

1. **La gestion critique du déséquilibre :** L'analyse initiale a montré que sans traitement spécifique, un algorithme classique ignorerait totalement les fraudes (0.17% du dataset). L'implémentation de la technique **SMOTE (Synthetic Minority Over-sampling Technique)** a été le facteur déterminant de la réussite du projet, permettant de rééquilibrer artificiellement les classes pour l'entraînement sans perdre d'information.
2. **La pertinence du Deep Learning :** Notre réseau de neurones artificiels (ANN) a démontré sa capacité à capter des non-linéarités complexes entre les 30 variables d'entrée. L'architecture choisie, intégrant des couches de régularisation (**Dropout**) et de normalisation (**BatchNormalization**), a permis d'obtenir un modèle stable qui ne souffre pas de sur-apprentissage (overfitting), comme en attestent les courbes de perte (Loss) convergentes.

3. **L'importance du prétraitement** : L'utilisation du **RobustScaler** sur la variable Amount a permis de neutraliser l'effet néfaste des valeurs aberrantes (montants très élevés), offrant au réseau de neurones des données d'entrée homogènes.

5.2 Analyse critique des résultats

Les performances du modèle, évaluées sur un jeu de test non modifié (reflétant la réalité), sont encourageantes mais appellent à la nuance :

- **Forces** : Le modèle atteint un **Rappel (Recall) de 84%**. Cela signifie que sur 100 fraudes tentées, le système en bloque 84. Dans un contexte bancaire, c'est la métrique prioritaire, car une fraude non détectée entraîne une perte financière directe et irréversible pour la banque ou le client.
- **Faiblesses** : La **Précision** tourne autour de **59%**. Concrètement, cela implique que lorsque le modèle lance une alerte, il se trompe près d'une fois sur deux (Faux Positif). Bien que moins grave qu'une fraude manquée, un taux de faux positifs élevé génère des coûts opérationnels (temps passé par le service client à vérifier les transactions) et peut dégrader l'expérience utilisateur (blocage de carte injustifié).

5.3 Perspectives d'avenir

Pour transformer ce prototype académique en une solution industrielle robuste, plusieurs pistes d'amélioration sont envisageables :

1. **Approche "Cost-Sensitive Learning" (Apprentissage sensible aux coûts)** : Actuellement, notre seuil de décision est fixé à 0.5. Nous pourrions optimiser ce seuil en intégrant une matrice de coûts réels. Par exemple, si une fraude coûte en moyenne 500€ à la banque et qu'une vérification client coûte 5€, nous pourrions ajuster mathématiquement le seuil pour minimiser la perte financière totale plutôt que de maximiser une métrique abstraite comme le F1-Score.
2. **Détection d'anomalies non supervisée** : Les fraudeurs innovent en permanence. Un modèle entraîné sur des fraudes passées (supervisé) pourrait rater de nouveaux types d'attaques. Il serait pertinent de coupler notre modèle avec un **Auto-encodeur (Autoencoder)**. Entraîné uniquement sur des transactions normales pour les reconstruire, cet algorithme signalerait comme anomalie toute transaction qu'il ne parvient pas à reconstruire correctement (erreur de reconstruction élevée), détectant ainsi des fraudes inédites.
3. **Modèles Séquentiels (RNN/LSTM)** : Si nous avons accès à l'identité des clients, l'utilisation de réseaux de neurones récurrents (LSTM ou GRU) permettrait d'analyser la séquence temporelle des achats. Cela permettrait de détecter, par exemple, une carte utilisée à Paris puis à New York en l'espace d'une heure.
4. **Mise en production et MLOps** : L'étape finale serait le déploiement du modèle via une API REST (FastAPI) conteneurisée (Docker), capable de répondre en moins de 50ms pour ne pas ralentir les paiements, avec un système de monitoring pour surveiller la dérive des données (Data Drift) au fil du temps.